

Inigo Quilez :: articles :: domain repetition - 2008, 2013, 2023



Intro

Raymarching your first SDFs (Signed Distance Fields) is a magical experience that all computer graphics practitioner should have, because realizing that just half page of code is sufficient to get some 3D object animated and rendered with shadows and all, without any engine or renderer's assistance, is indeed as magical as it gets in this field.

After this initial shock, a second mind blowing moment comes from learning about the **Smooth Minimum** function, which enables blending objects together effortlessly, and so sculpting organic shapes without having to worry about tessellation, topology reconnections and all the very tedious details that only skilled technical artists master.

And the third mind blowing moment arrives when one learns that with Domain Repetition a simple line of code can replicate an object infinite many times without requiring writing hundreds of lines of instancing infrastructure. And this is exactly what this article is about. We'll explain Domain Repetition, its strengths and weaknesses, and its applications. So get ready, here we go:



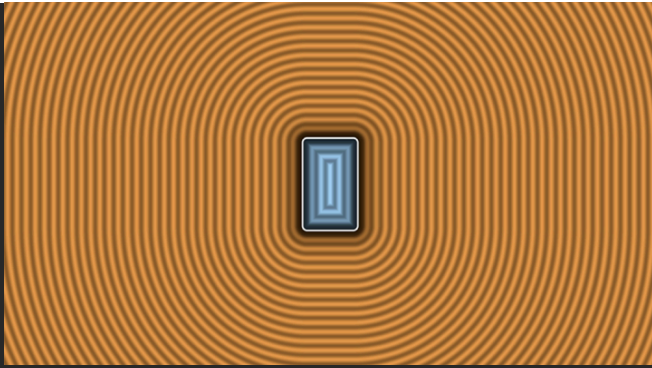
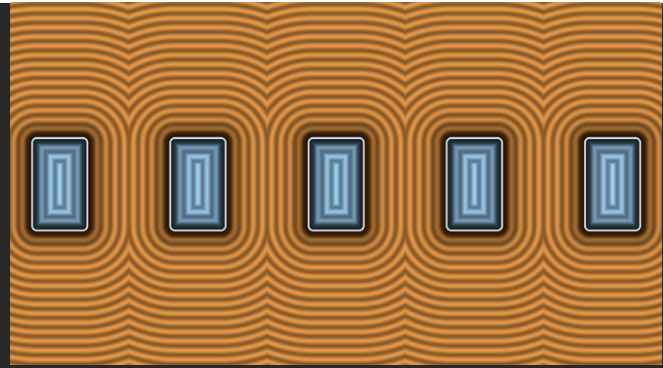
Domain Repetition creates infinite number of columns and floor tiles (2008)

The Idea

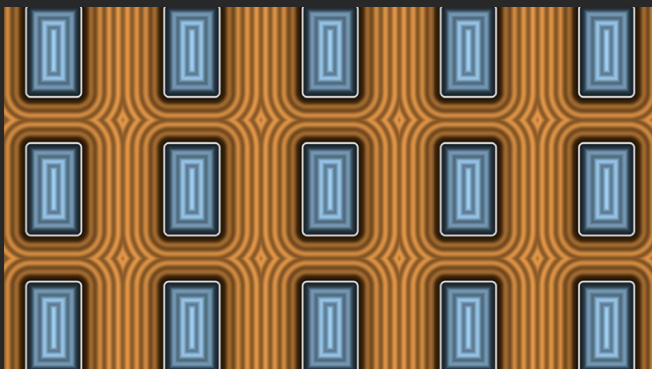
The basic idea of Domain Repetition is to take an SDF that is expressed as a mathematical function, and make it periodic - that is, repeat forever over space. Like a $\sin(x)$ function does. For example, one simple way to make an SDF repeat forever in the X direction is the following:

```
// repeat space
float repeated( vec3 p )
{
    p.x = p.x - round(p.x);
    return sdf(p);
}
```

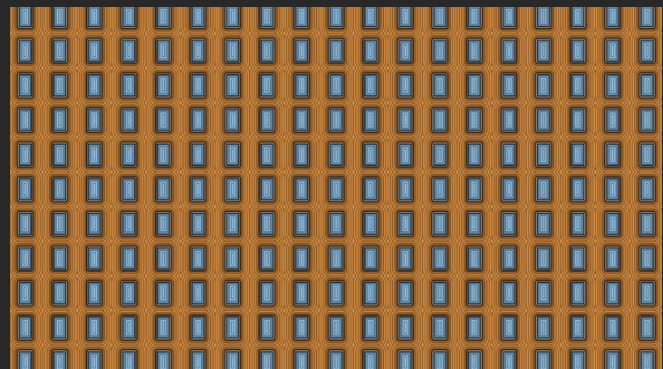
where **sdf()** is the basic shape that we want to repeat. Here the **round()** function is taking the closest integer to **x** and making it the new center of coordinates, effectively making a new coordinate system every 1 unit of distance. It's basically tiling space in the **x** direction. Or in other words, taking the domain of the function, **p**, and continuously resetting it to the range $(-0.5, 0.5)$. As long as SDF **sdf(p)** (a rounded box in the renders below) is defined within the range $(-0.5, 0.5)$, it will become periodic and repeat itself:

Basic SDF, $\text{sdf}(p)$ Domain Repetition, $f(\text{repeat}(p))$

Naturally multiple dimensions can be made periodic at the same time by simply applying the periodicity technique to each dimension, enabling for 1D, 2D or 3D repetition as seen below. Remember, just like a $\sin(x)$ wave, this repetition extends to infinity, giving us infinitely many copies of our geometry, even though we are only evaluating a single one! And I really mean there's a single evaluation of the rounded rectangle per pixel; no for-loops are involved here. Pretty cool.



Domain Repetition in 2D



Zoom out view of the previous

This is fantastic already, but we can make it even more powerful by allowing to control the spacing between the repetitions, the period of the function. This is easily done by scaling the domain before resetting the periodic coordinate system, and then compensating for it afterwards:

```
// repeat space every s units
float repeated( vec3 p, float s )
{
    vec2 r = p - s*round(p/s);
    return sdf(r);
}
```

Great. But we are not done yet, there are lots of fun things we can do with this technique.

Making it more powerful

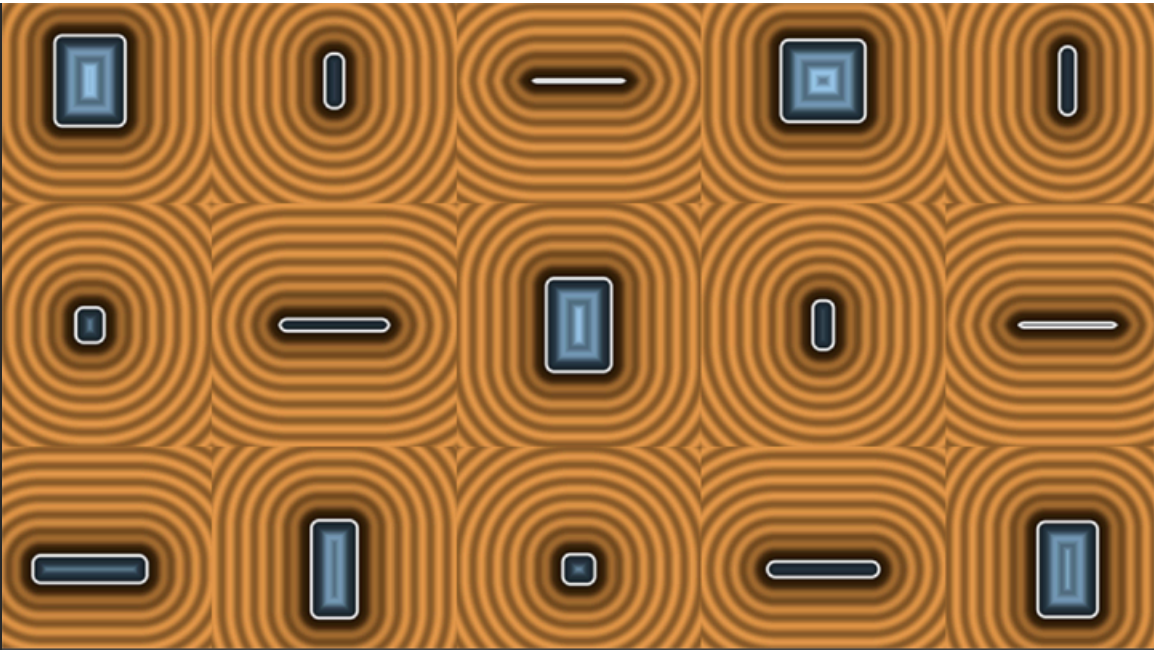
Now, you might have noticed that very few things in nature are perfectly regular and so nothing repeats itself exactly or at perfectly equal intervals. And so we probably want each one of our SDFs copies to be slightly (or radically!) different from each other. This is common practice in instancing systems as well, where one introduces size, color or orientation variations to each instance in an attempt to break regularity. In the picture at the head of this article I used Domain Repetition to make the columns and floor tiles (this was actually the painting that I used at the [NVscene 2008 event](#) to introduce the Domain Repetition technique!). Both columns and floor tiles used the instance **id** to create variations - a different erosion pattern (the holes) per columns and a different color and texture per floor tile.

So first thing to do now is to find a way to identify each instance within our infinite grid and assign it an instance **id**, a unique identifier. Luckily this is easy - note that the **round()** function was helping us snap our coordinate system to the closest integer in the domain. So, essentially, that integer is a unique identifier for any given copy of the SDF. If the domain is 2D or 3D, this identifier will be 2D or 3D, meaning we'll get one integer identifier per dimension. Getting a singular integer identifier from two or three interger coordinates is not difficult (hashing or just assuming some large but finite raster-like support grid).

For 3D, creating and using that identifier **id** would be as simple as rewriting the code above like this:

```
// repeat space every s units
float repeated( vec3 p, float s )
{
    vec3 id = round(p/s);
    vec2 r = p - s*id;
    return sdf(r, id);
}
```

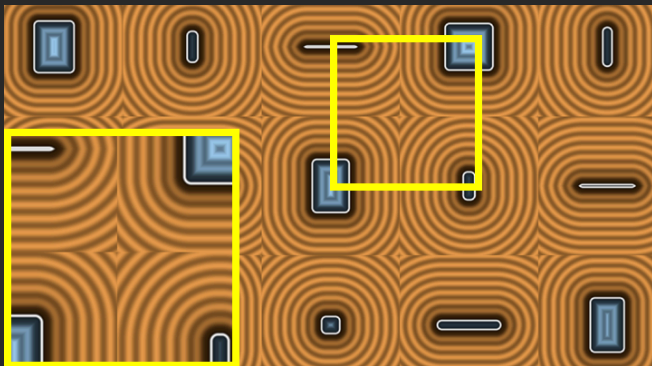
Note how the instance identifier **id** is passed to the SDF, so that it can use it to modify itself. For example, it could use it to change its own size:



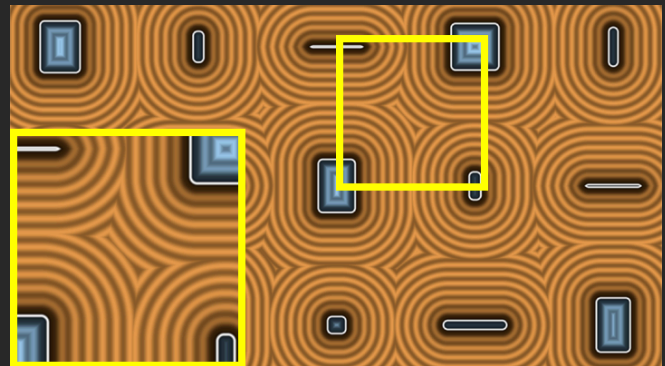
Using ID to modify size (naive)

In this case the exact math I used to modify the rounded boxes' size based on the `id` is not important for the discussion about Domain Repetition. But if you really are curious, in this case I made the size = $\text{vec2}(0.3, 0.2) + 0.3 * \sin(\text{id}.x * 111.1 + \text{id}.y * 2.4 + \text{vec2}(0, 2))$, where all the numbers inside the $\sin()$ function are totally arbitrary and have no particular significance, nor they are the best for the task - I very literally just used the first thing my fingers typed when I let them fall over the keyboard.

The interesting thing that we do need to talk about though, is that the this way to do the Domain Repetition is now incorrect, as you can see in the image above. If you haven't noticed any difference, pay more attention: the image has some discontinuities, sometimes the distance isolines don't match with each other. Below is a zoomed version of it, with one of the problematic areas highlighted in yellow. These discontinuities are one symptom of an incorrect SDF. Let me show you the correct SDF next to it, and pay attention to the isoline continuity in the highlighted areas:



Incorrect SDF



Correct SDF

This type of incorrectness might or might not be fatal for a given particular application, depending on how it uses the SDF. We'll talk about it shortly, but for now note how the second image with the same configuration of rounded boxes doesn't suffer from this problem - all isolines are continuous. Great. But what were these problems exactly and how were they fixed?

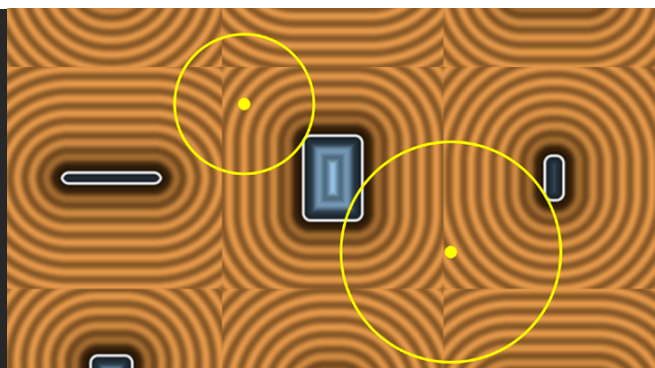
We've got a problem

So let's talk about what the problem actually is, before we talk about why it happens and how to fix it.

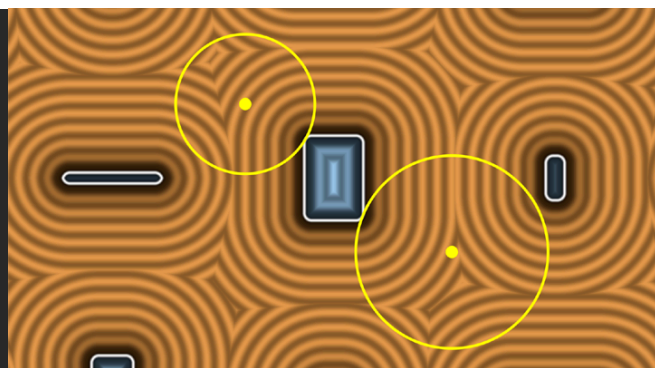
Remember that what we are trying to achieve here is to have a singular call to **sdf(p)** (the rounded box in these renders) and apply the **repeated()** modifier to its domain such that the result is also an SDF that contains infinite copies of **sdf(p)**. This SDF has to be an SDF indeed, that is, a (signed) field (function over space) that measures the distance to the closest surface, the surface of an infinitely many times repeated version of the original **sdf(p)**. And we almost got it right.

The problem is that the **round()** function we used to implement **repeated()** is assigning each point in the plane a local coordinate system, or a tile, if you want to see it that way. So each point only knows of one tile in the grid and therefore one instance of **sdf(p)**. This is precisely its main gift when it comes to performance, but it also means that if the closest instance is not in the same tile than the evaluation point **p**, then this technique will fail.

This can happen for a variety of reasons, and can only be fully avoided if all the instances are exactly the same size, shape and orientation, or they are symmetric with respect to the tile boundaries (more of this later). But in general, a neighboring tile might host a very large instance compared to the one in the current tile, so the closest instance is actually in that neighbor tile, not in the current one hosting **p** (the exact condition is of course that the difference in sizes is smaller than half the width of the tile). This is exactly what's happening in the diagram below:



Incorrect Domain Repetition



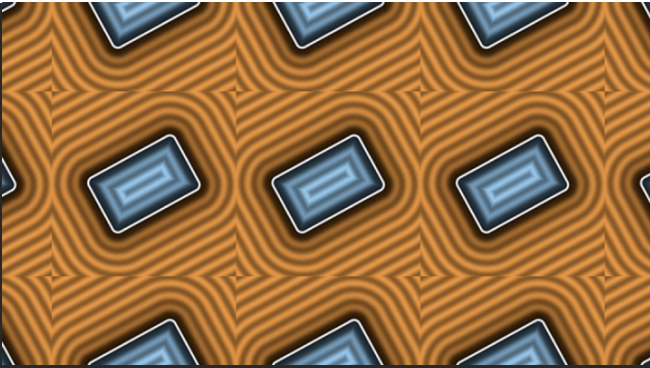
Corrected Domain Repetition

Here in the first image we have again our naive implementation of Domain Repetition as seen in the code above, and I have highlighted two yellow points where we are measuring distances, enclosed in a circle that indicates what the distance value is at that point. As you can see, the top left point looks just fine: its circle touches exactly the rounded box **sdf(p)** at the center of the tile it belongs to. This means when we evaluated **repeated(sdf(p))** we did get the correct value - the distance to the closest surface.

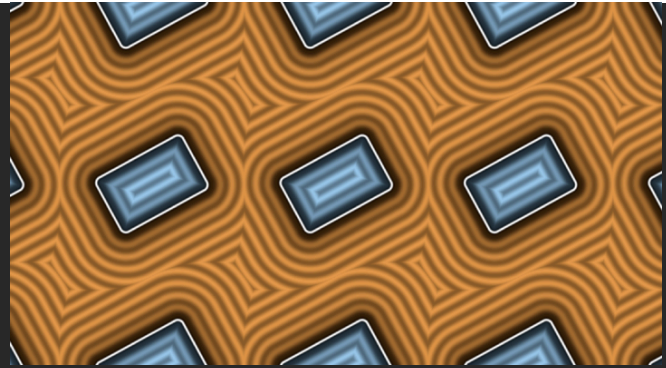
However things don't look right at the second yellow point, because the circle is touching the rounded box that is in its own tile, but in doing so is penetrating the larger rounded box instance on its left neighboring tile. So the circle is larger than it should have been, for it was supposed to only just touch the closest surface, wherever it is. In fact, if this 2D rounded boxes where the profile of 3D columns of some sort of temple and we were rendering it with a raymarcher, which explicitly depends on this closest surface measurement for correct functioning, we'd be seeing artifacts in the form of holes and cracks in the columns as the camera moved around the scene. A collision detection system based on SDFs would also fail in this case.

On the second image you can see what the correct SDF should have looked like, again, and how the yellow circles at the same sampling points do behave as expected.

This problem of incorrect distances also happens when you do not modify the size of the instances but just rotate them (a lot like in a Truchet pattern, or even just a little like in the picture below). Or when you shift or jitter them (at which point we've just generalized of a traditional **Voronoi** construction). Or in general when you perform any operation that breaks symmetry along the tile boundaries. Here's an image of a very small rotation of the rounded box base instance, where we can see how the SDF is broken again. The following image shows what the SDF should have looked like instead:



Incorrect Domain Repetition



Corrected Domain Repetition

Fixing the problem

We know that when we are evaluating our SDF at point $\mathbf{p}$ through our `repeated(sdf(p))` function, it is not sufficient to consider the instance at the center of the current tile or local coordinate system, because the closest shape could be in a neighboring tile. So, all we need to do is check the neighboring tiles and evaluate `sdf(p)` there and find the closest distance to any of them. When the objects are large, this can be a lot of cells. But if the objects are comparable to the size of the tiles, then the scan area is only a single ring of tiles. On top of that, we can be a bit smart about it and note that we only need to explore the neighbors that do have a chance of hosting an instance that is closest than the current one. And those are only the neighbors that are on the closest tile sides to our sample $\mathbf{p}$. And so, in 2D we don't need to check $3 \times 3 = 9$ tiles as we'd expect (current + 8 neighbors), but only $2 \times 2 = 4$ (current + 3 neighbors). In 1D we only have to sample 2 tiles (current + 1) rather than 3, and in 3D only $2 \times 2 \times 2 = 8$ (current + 7) rather than $3 \times 3 \times 3 = 27$. As mentioned, if the objects span more than a tile, we obviously need to inflate this search area appropriately.

For the examples in the images above, the code for 2D would be like this (Shadertoy example here:

<https://www.shadertoy.com/view/dt2czK>):

```
// correct way to repeat space every s units
float repeated( vec2 p, float s )
{
    vec2 id = round(p/s);
    vec2 o = sign(p-s*id); // neighbor offset direction

    float d = 1e20;
    for( int j=0; j<2; j++ )
        for( int i=0; i<2; i++ )
```

```

{
    vec2 rid = id + vec2(i,j)*o;
    vec2 r = p - s*rid;
    d = min( d, sdf(r) );
}
return d;
}

```

This code produces the correct SDF, as shown in the right image at the end of the previous section. Here the **sign()** function takes care of determining whether the left or right neighbor needs to be checked for shape proximity (or top vs front and up vs down). Please note that if the base instances can be large enough to span more than a tile, then we might need to check a larger neighborhood of tiles.

A different fix to the problem

The method above works well, and in the big scheme of things, it is not very expensive - we have increased the cost of computation *only* by 4x in 2D and 8x in 3D, which is not bad at all given that we still have infinite many copies and variations of them, not just 4x or 8x more. We are still evaluating all this infinite complexity in onstant time!

But of course an 8x figure can still be the difference between an interactive application and a useless one, or between getting feedback on your art once a week rather than every day. So we'd like to have some alternative way to fixing the discontinuity issues in the first and most naive implementation of **repeated()** we presented by only scanning a single tile per evaluation again.

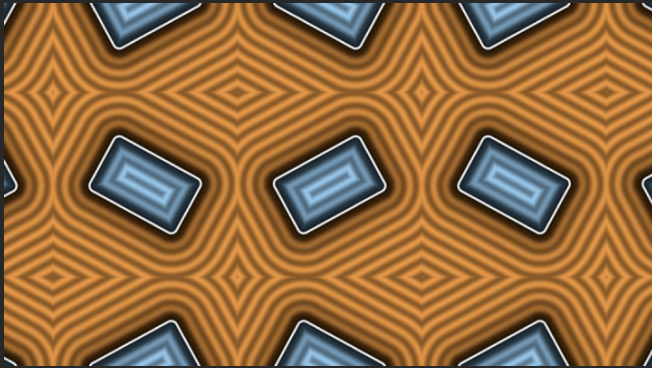
And luckily for us, there is something that we can do. Remember the problem is the asymmetry across tile boundaries, so perhaps we can fix *that*. I first saw Shadertoy user and demoscener Fizzer do it: force symmetry by mirror objects every other instance, such that the closest surface is always guaranteed to be in the current tile. Something like this:

```

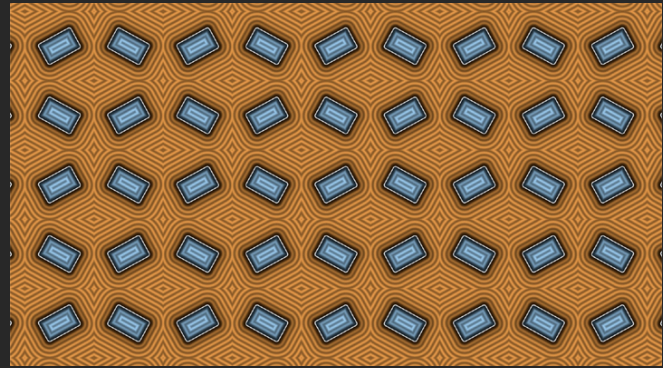
// fast space repetition by mirroring every other instance
float repeated_mirrored( vec2 p, float s )
{
    vec2 id = round(p/s);
    vec2 r = p - s*id;
    vec2 m = vec2( ((int(id.x)&1)==0) ? r.x : -r.x,
                  ((int(id.y)&1)==0) ? r.y : -r.y );
    return sdf( m, id );
}

```

This naturally won't be a valid solution in all occasions from an artistic point of view, but it can be in some cases, so it is a great tool to have in your SDF toolbox.



Mirrored repetition fixes discontinuities



Zoomed out view

Rotational and Rectangular Repetition

A natural extension of the technique is to apply Domain Repetition to any domain parametrization or transformation. A simple example is using polar coordinates (angle, distance) rather than the regular cartesian coordinates (x and y), and make the angular coordinates periodic so objects repeat in a ring configuration. We could call this "angular" or "rotational" repetition (not "radial" since that would be a straight line configuration). For example, if we want n repetitions of our basic SDF **sdf(p)**, we can do this:

```
// rotational/angular repetition
float repetition_rotational( vec2 p, i
{
    float sp = 6.283185/float(n);
    float an = atan(p.y,p.x);
    float id = floor(an/sp);

    float a1 = sp*(id+0.0);
    float a2 = sp*(id+1.0);
    vec2 r1 = mat2(cos(a1),-sin(a1),si
    vec2 r2 = mat2(cos(a2),-sin(a2),si

    return min( sdf(r1,id+0.0), sdf(r2
}

```



Rotational Repetition -

<https://www.shadertoy.com/view/XtSczV>

This application is a 1D domain repetition, since we are only doing the angle periodic, and so we need to check and evaluate the basic SDF **sdf(p)** twice. If you ever tried a simpler version of rotational repetition that only evaluates the instance **sdf(p)** once, now you know why you were getting rendering artifacts!

The code above is not too complicated, but we can dissect it a bit. In the first one we compute the repetition spacing, which is the full circle divided by the number of copies we want. In the second line we compute the angular coordinate of space, which we'll make periodic. Then we compute the **id** of the tile with a **floor()** function. We could also use the **round()** and dynamic offset technique shown in previous examples, but I like to show this variant as well. Then we take each of the two tiles and rotate them to the first tile, which effectively creates a new coordinate system for it. Then we evaluate the base SDF at the current and neighbor tiles, and we take the minimum distance of the two. Note we still pass the tile **id** to **sdf(p)** so we can do per instance variations. You can find this code and realtime demo in Shadertoy here:

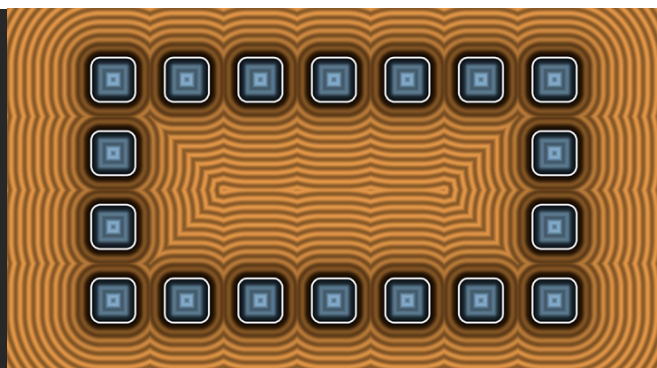
<https://www.shadertoy.com/view/XtSczV>.

Also note that if you really want to avoid trigonometrics, see this shader as an example of 5-fold rotation through reflections instead of trigonometric functions: <https://www.shadertoy.com/view/lscCR8> ; although if you are using a GPU then you probably should unlearn to avoid trigonometrics anyways, because they are not nearly as slow compared to multiplications as they are in CPU architectures.

As I said, the repetition can be applied to any parametrization that you can come up with. For example this is a repetition along the perimeter of a rectangle, which Shadertoy user "timestamp" improved over my original implementation and can be found here: <https://www.shadertoy.com/view/ctjyWy>:

```
// rectangular repetition - ONLY WORKS FOR SYMMETRIC SHAPES
float repeat_rectangular_ONLY_SYMMETRIC_SDFS( vec2 p, vec2 size, float s )
{
    p = abs(p/s) - (vec2(size)*0.5-0.5);
    p = (p.x<p.y) ? p.yx : p.xy;
    p.y -= min(0.0, round(p.y));
    return p*s;
}
```

Please note the above only checks for **one** instance of **sdf(p)**, so it will suffer from the artifacts we've been talking about if the shape of **sdf(p)** is not symmetrical. This particular type of repetition can actually be useful when describing the columns of a Greek temple:



Rectangular Repetition (7x4)



Temple columns use Rectangular Repetition (9x5):

<https://www.shadertoy.com/view/ldScDh>

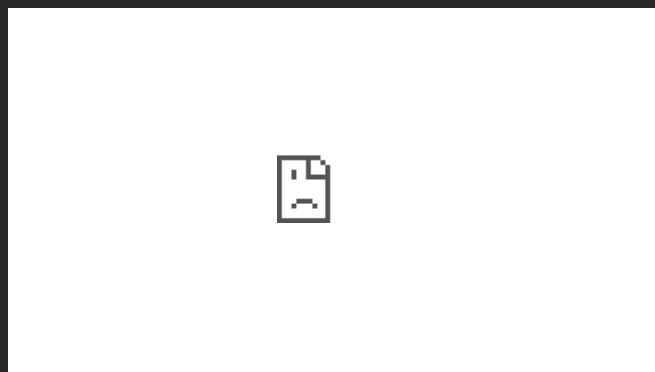
And this is a variation of the above that extends to infinity: <https://www.shadertoy.com/view/tcVBDw>

Other parametrizations

As I said, any domain parametrization can be made repeating, so I'll just leave a few other options here for you to explore following the links below, including a Fibonacci repetition over a sphere as described by Sanger et al in [this paper](#) and exemplified in [this shader](#) and more artistically utilized in the "Fall" shader below. And also a recursive repetition pattern of negative boxes that construct a Menger sponge fractal, as described in [this other article](#):



<https://www.shadertoy.com/view/lltBWB>



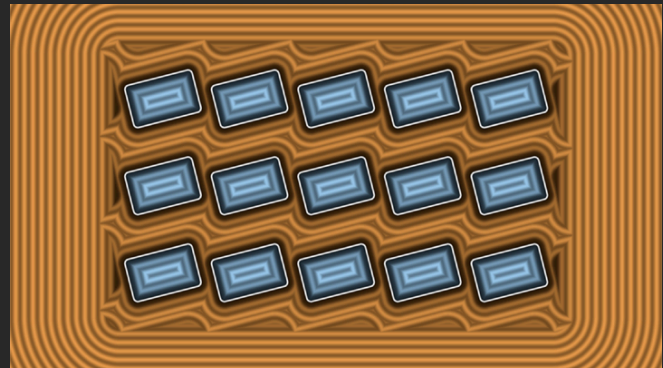
Limited repetition

All these repetition variations are great and fun, but the truth is that, in practice, the Domain Repetition you'll find yourself using most often is the plain, cartesian grid repetition we begin the article with. This is because we humans tend to build things following such regular patterns. However, unlike our original Domain Repetition pattern we discussed so far which extends to infinity, like a $\sin(x)$ wave, the repetition of elements in human objects doesn't go forever. Instead we usually repeat things a given number of times, for example 88 keys in a piano, or 4x10 windows in the facade of a building.

So, in order to use our infinite grid repetition technique in practice we should limit its extent, somehow. The naive way to do so is to perform a boolean intersection of the infinite grid with a container box of the required dimensions, so we can disable all the instances that we are not interested in. Such intersection of SDFs is usually performed with a `max()` operation, as described in my [basic SDF article](#), so the code could look something like this:

```
// naive limited/finite repetition
float limited_repeated( vec2 p, vec2 s
{
    vec2 id = round(p/s);
    vec2 o = sign(p-s*id);
    float d = 1e20;
    for( int j=0; j<2; j++ )
    for( int i=0; i<2; i++ )
    {
        vec2 rid = id + vec2(i,j)*o;
        vec2 r = p - s*rid;
        d = min( d, sdf(r) );
    }

    vec2 q = abs(p) - (size*s*0.5);
    float w = min(max(q.x,q.y),0.0)+1e
    return max( d, w );
}
```



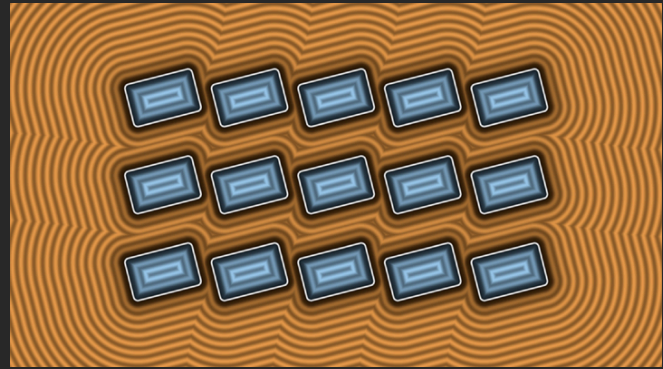
Naive Limited Domain Repetition

Here the first block of code is exactly the same as in the infinite domain repetition implementation we designed earlier on the article. The last three lines of code implement the sdf of the container box (dimensions, in instances, given by the parameter `size`). Don't worry about understanding the implementation of this box SDF, its workings are irrelevant to the discussion (although feel free to check [this Youtube tutorial](#) if really interested). The last line computes the boolean intersection of the infinitely repeated SDF and the container box with the `max()` function, so that in its exterior the repeated SDF disappears.

The image to the right of the code is created by that code with a size of (5,3) and the **sdf(p)** of a rounded rectangle as base instance. Now, while it successfully got rid of the infinitely many instances outside the 5x3 container box, your SDF trained eye should realize that the SDF is actually incorrect. The isolines are continuous, but funky. Indeed, this SDF will again produce incorrect shadows and collision detection, although correct raymarched intersections this time since this particular type of incorrectness produces distances that are always smaller than the real distances.

The solution is surprisingly simple - instead of doing the boolean in 2D/3D space, we do it in the **id** values of the instances. What I mean is that we just go with the infinite or regular Domain Repetition technique, but check whether the instance identifier (**id**) is within the range of valid instances. In the previous example, since the grid we want is 5x3 instances, we want our instance **id** to be within the range [-2,2] in the X axis (making a total of 5 instances: -2,-1,0,1,2) and within the range [-1,1] in the Y axis (making a total of 3 instances: [-1,0,1]). If an **id** is outside that range, we clamp it/limit it to our valid range:

```
// corrected limited/finite repetition
float limited_repeated( vec2 p, vec2 s
{
    vec2 id = round(p/s);
    vec2 o = sign(p-s*id);
    float d = 1e20;
    for( int j=0; j<2; j++ )
    for( int i=0; i<2; i++ )
    {
        vec2 rid = id + vec2(i,j)*o;
        // limited repetition
        rid = clamp(rid, -(size-1.0)*0.
        vec2 r = p - s*rid;
        d = min( d, sdf(r) );
    }
    return d;
}
```



Correct Limited Domain Repetition

As you can see all we needed compared to the original infinite repetition is a single line of code. For an even number of instances you might want to offset the technique by half spacing and change the clamp limits accordingly. Regardless, this technique looks even neater when we know the shapes are symmetry and we can get away with doing a single SDF evaluation:

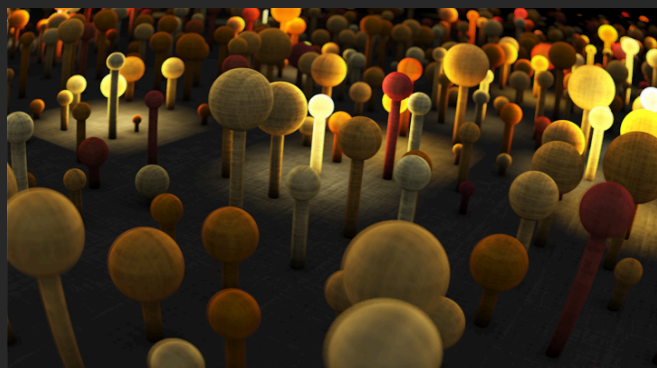
```
// limited repetition - ONLY WORKS FOR SYMMETRIC SHAPES
float repeated_ONLY_SYMMETRIC_SDFS( vec3 p, float s, vec2 lima, vec2 limb )
{
    vec3 id = clamp( round(p/s), -lima, limb );
    vec2 r = p - s*id;
```

```
return sdf(r, id);  
}
```

You can see the reference code in action in here: <https://www.shadertoy.com/view/3syGzz>, where you can move the mouse and explore the distance errors.

Applications

Let's close this article by showing some applications of domain repetition. Here on the first image we see an infinite 2D Domain Repetition, where the id of the instance is used to randomize the height and color for the light emitting lanterns. On the second one, infinite 3D Domain Repetition is in action, with the instance id used to offset the phase of the animation cycle that wings the angel's wings.



<https://www.shadertoy.com/view/4sB3D1>



<https://www.shadertoy.com/view/lssGRM>

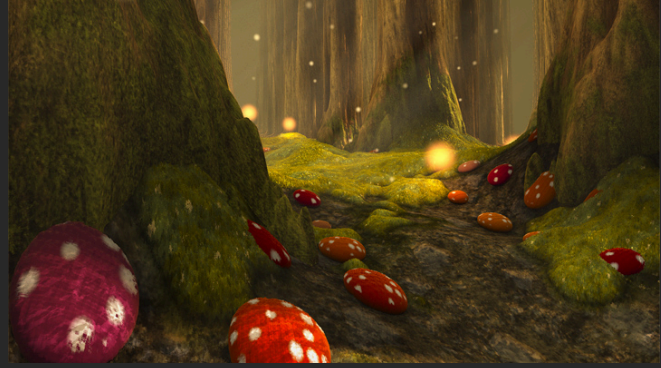
In the following images, first I used three levels of Domain Repetition in order to construct the moss plants: first a basic 2D domain repetition was used to create many different plants. Each plant had randomized size and orientation based on its instance ID. The plants themselves were made of a line segment for a stem with many petals around it. In reality a single petal existed of course, and 1D lineal domain repetition was used to instantiate it vertically along the stem, and then rotational repetition was used for each one of those to populate all sides of the plant. There also 3D Domain Repetition applied to a single sphere in order to make the white floating particles.

And on the second image, infinite 2D repetition was used to create a forest out of a single tree and a single mushroom with a single white dot on it. The id was used to color each mushroom differently. Similarly, the little dust particles floating in the air was done with 3D repetition, with the id controlling

the always unique size and motion of each particle.



<https://www.shadertoy.com/view/4tByz3>



<https://www.shadertoy.com/view/XsfGD4>

In the piano below, limited 1D repetition was used to create the keys of the piano, and its pedals. On the right side, 2D domain repetition was used to create the freckles in the girl's face from a singular brown dot, and the forest in the background from a single tree, and a 1D limited domain repetition for creating the branches inside each pine tree, and the blades and posts of the bridge from a single one.



<https://www.shadertoy.com/view/ldl3zN>



<https://www.shadertoy.com/view/WsSBzh>

Lastly, in the abstract landscape on the first image, the background blobs are done with 2D domain repetition, including having a different animation offsets based on the instance id, and so are the candy balls on the floor which their own individual color and position offset. On the second image, the rainforest is made of infinitely repeated ellipsoidal trees (to learn more you can watch this Youtube tutorial I made on this particular painting: <https://www.youtube.com/watch?v=BFId4EBO2RE>):



<https://www.shadertoy.com/view/3lsSzf>



<https://www.shadertoy.com/view/4ttSWf>

As you can see, Domain Repetition comes really handy when creating scenes with purely procedural SDFs, by enabling an infinite amount of detail in a very efficient way (runtime cost, memory, and storage). This article doesn't cover all flavors and variations of the technique, but should give you a solid understanding to start making your own new applications of repetition. Have fun!

inigo quilez - learning computer graphics since 1994